CellForge: Language-Model Agents as Direct Authors of Neural Networks

First results on gradient-free construction of an OCR model

CellForge project*

Draft, 29 September 2026

Abstract

We ask whether large language model (LLM) agents can build useful neural networks by *directly writing* their weights, biases and wiring, with no gradient-based fitting of the resulting artifact. CellForge gives an orchestrator LLM and scoped *maker* agents a recursive graph representation (primitive cells composed into arbitrarily nested, shareable bundles with optional conditional dispatch), atomic validated transactions, a versioned development harness, and an append-only evidence record. Final evaluation runs on a separate machine under a separate account with freshly generated cases for every evaluation. In a first round on variable-length digit-string recognition, four independently authored artifacts—including one written with no metric feedback at all—read in-distribution final cases perfectly (exact match 1.000, $n = 2000$). Under distribution shift, they diverge: the most robust artifact reads 73.8% of strings in an unseen font and 72.3% under size shift, while the others reach 38–57%. A gradient-trained slot-MLP control reaches 83.8% in distribution but 10.0% on the unseen font; random direct-edit search reaches 0%. These results establish that LLM authors can construct working visual recognizers without gradient training; they do not yet establish robustness, scaling, or superiority over well-designed gradient-trained models.

1 Introduction

Neural networks are almost always *fitted*: an optimizer adjusts parameters to reduce a loss on data. LLMs, however, can read images, reason about mechanisms, and write precise numerical specifications. This suggests a different construction process: an LLM *authors* a network the way an engineer writes a program, observes its failures, and revises it. The author, not gradient descent, is the optimizer.

CellForge is an experimental system for testing this idea rigorously. Its primary question is:

> Can LLM agents discover useful neural computations, their recursive organization, and better ways of developing them, using evidence-backed memory and direct numerical and structural edits, producing an offline image-to-text artifact that improves on independent evaluation?

Scope and non-goals. CellForge tests whether LLM agents can author neural network weights directly, without gradient training. The artifact studied here is an offline image-to-text recognizer. It has no language understanding, dialogue, speech capability, or general intelligence; chat and speech interfaces are out of scope for this work.

*Orchestrator and maker agents: Claude (Anthropic) running in Claude Code. Project owner: [name to be supplied].

Motivation and long-term aim. The project owner's long-term aim is more ambitious: to work toward general intelligence, eventually a system one can converse with, with visual input and later speech, whose computation is constructed by LLM authors rather than fitted. We state this aim to explain the design choices (modality-agnostic decoder, recursive reusable components, staged vocabularies), and we keep it strictly separate from our claims. Section 7 lists what would need to be demonstrated, in order, for that aim to become plausible, and the risks at each stage. Nothing in the present results bears on general intelligence.

Contributions. (1) A recursive cell/bundle IR with lazy conditional dispatch, content hashing, and behaviour-preserving wrap/unwrap refactors; (2) a transaction and governance design that makes every parameter change traceable to an accepted LLM proposal and separates development feedback from independent final evaluation; (3) first measurements of LLM-authored OCR artifacts against gradient-trained and random-search controls, including distribution-shift tests.

2 Related work

Architecture search composes motifs hierarchically [1] and evolves topologies and components [2]; both fit weights by training. Our evidence memory adapts ideas from visual retrieval-augmented generation [3, 4, 5]. We are not aware of prior work in which LLMs author all weights of a working recognizer without fitting, but a wider prior-art review is required before any novelty claim.

3 System

3.1 Representation

A *cell* computes $h = \sigma(W_1 x + b_1)$, $y = W_2 h + b_2$ over the last axis of its input. A *bundle* is a typed DAG whose nodes are parameter-free operations, parameter tensors, cells, other bundles, or a top-1 `dispatch` among bundles; unselected branches do not execute. The same representation is used at every depth; definitions may be shared, and parameter sharing is explicit. Recursive template references and exponential expansion are rejected at validation. A model has an `encode` bundle (pixels $\rightarrow$ memory) and a `step` bundle (memory, step index, previous token, state $\rightarrow$ logits, state), driven by a bounded greedy decoder that stops on the model's own EOS.

3.2 Construction protocol

Makers submit JSON proposals of atomic operations (set parameter by explicit value or by a deterministic expansion of an author-written pattern such as ASCII art; put/delete definitions; wrap/unwrap). Proposals are validated against their exact base hash, executed on sample inputs, and, for pure refactors, tested for exact behavioural equivalence. Makers may not run code that computes weights from data, may not script search loops, and receive development feedback only through a budgeted `dev-eval` command.

3.3 Evaluation boundary

Development data, a fixed reference anchor, and all diagnostics are author-visible and therefore development feedback. Final cases are rendered on a separate server by an `evaluator` account with a fresh secret seed per evaluation, excluding (for strings of length ≥ 4) every string author

Artifact	iid	unseen font	size shift	length 9
Authored, iterative A	1.000	**0.738**	**0.723**	1.000
Authored, iterative B	1.000	0.382	0.379	0.000
Authored, one-shot	1.000	0.427	0.453	1.000
Authored, flat	1.000	0.407	0.570	1.000
Gradient slot-MLP, 400	0.057	0.054	0.050	0.000
Gradient slot-MLP, 20k	0.838	0.100	0.419	0.000*
Random edits, 20k evals	0.000	0.000	0.000	0.000

Table 1: Exact-match rate on independent final cases (fresh seeds). Wilson 95% intervals are ± 0.03 or narrower at these n; CER in the project log. *The slot-MLP has eight character slots by construction.

agents ever saw; only aggregates leave the machine. The unseen-font split uses a font file that is never present on the authoring machine.

4 Experiment 1: digit strings

Protocol. 32×128 grayscale, 1–8 digits, DejaVu Sans Mono 20 px, horizontal offset 2–14 px, vertical jitter ±2 px, Gaussian noise $\sigma \leq 0.03$. Final splits: iid ($n = 2000$), unseen font Liberation Mono ($n = 1000$), size 18–22 px ($n = 1000$), length 9 ($n = 500$).

Arms. From an EOS-constant bootstrap: two independent iterative makers with recursive bundles (25 dev-eval budget each), a flat-graph maker (no sub-bundles), and a one-shot maker (validation only, no metrics). Controls: a gradient-trained slot-MLP (Adam, 400 or 20,000 training images) and (1+1) random direct-edit hill climbing (25 or 20,000 evaluations), all scored through the same reference evaluator.

Results. Table 1. All four authored artifacts are perfect in distribution, using two or fewer development evaluations each; the in-distribution task is therefore saturated and does not discriminate construction arms. Every maker converged on the same family of mechanism—vertical alignment, left-edge detection, a fixed 12 px character pitch, and hand-typed 15×12 glyph templates—yet small design decisions produced large robustness differences: artifact A's normalized agreement score and ±3 px search tolerance generalized best; artifact B hard-coded eight character cells and failed every 9-digit string.

What the results do and do not show. They show that LLM authors can construct compact recognizers (about two thousand parameters for artifact A) without gradient fitting, and that the one-shot author needed no feedback to reach perfect in-distribution accuracy. They do *not* show a difference between arms (one construction run per arm), robustness to real-world text, or superiority over gradient training in general: our control lacks translation invariance, and a gradient-trained convolutional network would likely be perfect in distribution. The makers' templates came from *looking at* development images; we regard the author's visual perception as the channel through which learning happens in this system, and we report it as such rather than as discovery from first principles.

Model	bpc	top-1	gramm.	gramm.+novel	experts	params
Ladder L1	0.6796	.810	.966	.788	6	278k
Ladder L2	**0.6776**	.809	**.997**	.884	60	1.69M
Ladder L3	0.6790	.809	.940	.804	4	898k
Open O1	0.6797	.809	.955	.839	–	1.31M
Open O2	0.6808	.809	.939	.775	–	1.56M
Gradient MLP (window 8)	0.824	.780	.201	.135	–	74k
Trigram counts	1.383	.617	0	0	–	66k
Uniform	5.000	0	0	0	–	32
Generator entropy: ≤ 0.6734 bpc (summed choice bits, upper bound)						

Table 2: Independent final evaluation on 1,000 fresh documents (bootstrap 95% intervals on bpc are about ±0.002). "Gramm." is the fraction of complete generated sentences accepted by the private grammar; "+novel" additionally requires the sentence to be absent from the authors' training text.

5 Experiment 2: hierarchical mixture-of-experts language models

The project owner judged digit recognition too easy a test of the intended cell→cluster→expert design, so the second experiment asks authors to build a character-level language model, with routing and recurrent state doing real work.

Task. Text comes from a private English-like grammar (12 nouns, 12 verbs, agreement that must survive relative clauses such as "the cat that the dogs see runs", adjectives, prepositions, adverbs, ", and" coordination) held only on the evaluation server. Authors see 2,000 sampled documents and 300 development documents, never the grammar. Vocabulary: 32 tokens. The model receives the last eight characters, the position, and its own state, and outputs next-character logits. Scoring is bits per character (bpc) under teacher forcing on fresh documents, plus the grammaticality of 200 free-running samples (temperature 0.8) judged by the private recognizer. Authors may read text but may not run code that counts or estimates anything from it.

Arms. Three independent authors in an enforced ladder profile (cells only inside clusters, clusters inside experts, a router whose scores drive a lazy top-1 dispatch among at least two experts, checked by a validator), and two authors in an open profile with any organization and optional routing. Controls fitted from data on the same 2,000 documents: uniform, n-gram count models, and a gradient-trained MLP over the same eight-character window (Adam, 74k parameters).

Results. Table 2. All five authored models reach 0.678–0.681 bpc, within 0.004 bpc (under 1%) of the generator's entropy, with 94–99.7% grammatical generations, against 0.824 bpc and 20% for the gradient MLP. The authored models keep singular/plural state across nested clauses in a stack held in recurrent state; the fixed-window MLP cannot. The best model (L2) has 60 routed experts, one per grammatical situation, and only 3 of 60 sampled sentences violate the grammar.

Interpretation and limits. The task is exhausted: every author read roughly 40 documents, wrote the closed grammar down, hand-tallied rates, and compiled it into tables and state machines. This shows the machinery (state, recursion tracking, dispatch among tens of experts) works end to end at 0.3–1.7M authored parameters. It does *not* show that the ladder helps: an open, unrouted model (O1) matched the ladder with three feedback runs against five to eleven, and the ladder

authors were required to use routing the task does not need. It also does not show that authoring scales to data whose generator cannot be read off from a few dozen samples; the next experiment moves to real text. Authoring cost (author-reported): 176–231k tokens per run.

6 Limitations

Single font family in development; one run per arm; subagent token usage is self-reported; authors and orchestrator share an operating-system account locally (the final-evaluation boundary is physical, on a separate server, but the orchestrator has administrative access to it); the live visual-retrieval memory arm and harness-evolution experiments are not yet run.

7 Roadmap beyond OCR

The path from this result toward the long-term aim requires, in order: robust alphanumeric recognition across unseen font families; real-world text lines competitive with a parameter-matched gradient-trained CNN (the critical go/no-go test, since authoring effort scales with case diversity rather than data); constructed sequence priors; measured component reuse across visual tasks; constructed algorithmic reasoning with length generalization; and only then language. The largest risks are scale (capable language models have 10^8–10^{12} parameters, far beyond hand authoring, pushing toward weight *generators*) and provenance (linguistic competence in an authored model would be transferred from the author LLM's pretraining and must be reported as such).

References

[1] H. Liu et al. Hierarchical representations for efficient architecture search. arXiv:1711.00436, 2017.

[2] R. Miikkulainen et al. Evolving deep neural networks. arXiv:1703.00548, 2017.

[3] S. Yu et al. VisRAG: Vision-based retrieval-augmented generation on multi-modality documents. arXiv:2410.10594, 2024.

[4] Y. Sun et al. VisRAG 2.0: Evidence-guided multi-image reasoning in visual retrieval-augmented generation. arXiv:2510.09733, 2025.

[5] Q. Wang et al. VimRAG: Navigating massive visual context in retrieval-augmented generation via multimodal memory graph. arXiv:2602.12735, 2026.